

Теоретический конспект №6

Тема: Два типа value-based методов

Связано с: [note 4.md](#) — Policy-Based vs Value-Based методы · [note 5.md](#) — Deep RL

1. Напоминание: зачем нужны value-based методы

В *value-based* подходах агент **не учит политику напрямую**, как в *policy-based* методах. Вместо этого он учится **оценивать "ценность" состояний или действий**, а потом **выбирает наилучшее**.

Мы не говорим агенту: «делай так», мы учим его понимать: «насколько хорошо находиться в этом состоянии и что стоит сделать дальше».

2. Определение value-функции

Value-функция отображает состояние (или состояние+действие) в ожидаемое **дисконтированное вознаграждение** (expected discounted return):

$$V_{\pi}(s) = \mathbb{E}_{\pi} \left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s \right].$$

где:

- $V_{\pi}(s)$ — ценность состояния s при политике π ;
- $\gamma \in [0, 1)$ — коэффициент дисконтирования;
- $R_{t+1}, R_{t+2}, \dots$ — будущие награды.

3. Связь между политикой и value-функцией

В *value-based* методах политика **не обучается напрямую**, но **всё равно существует**. Обычно это **жадная политика (greedy policy)**:

$$\pi(s) = \arg \max_a Q(s, a)$$

то есть агент **действует на основе ценности Q** , выбирая действие с максимальным значением. Таким образом:

обучая value-функцию → мы получаем оптимальную политику.

4. Два типа value-функций

1. State-value function (V-функция)

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Интерпретация:

«Ожидаемое суммарное вознаграждение, если я начну в состоянии s и буду следовать политике π ».

Пример: В лабиринте *FrozenLake*, если агент находится на клетке s_5 , то:

- $V(s_5) = 0.72$ означает, что начиная с этой клетки, агент в среднем получит вознаграждение 0.72 (с учётом вероятностей и дисконтирования).
-

2. Action-value function (Q-функция)

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Интерпретация:

«Ожидаемое суммарное вознаграждение, если я в состоянии s выполню действие a , а затем продолжу следовать политике π ».

Пример: Если агент стоит на клетке s_5 и может пойти:

- вправо: $Q(s_5, \text{вправо}) = 0.75$,
- вверх: $Q(s_5, \text{вверх}) = 0.32$,
- вниз: $Q(s_5, \text{вниз}) = -0.1$.

Тогда **оптимальное действие**:

$$a^* = \arg \max_a Q(s_5, a) = \text{вправо}.$$

5. Почему два типа?

Функция	Что оценивает	Применение
$V(s)$	ценность состояния	когда политика фиксирована
$Q(s, a)$	ценность пары состояние-действие	когда агент выбирает действие

Q-функция более гибкая, потому что напрямую отвечает на вопрос: «Что делать сейчас?» Именно поэтому **Q-Learning строится на $Q(s, a)$** .

6. Как из value-функции получить политику

Даже если политика не обучается явно, она **необходима** для действий агента. Есть два стандартных варианта:

Greedy Policy

$$\pi(s) = \arg \max_a Q(s, a)$$

Выбираем **наилучшее** действие (максимизируем Q).

ϵ -Greedy Policy

$$\pi_\epsilon(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{если } a = \arg \max_a Q(s, a) \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{иначе} \end{cases}$$

— добавляем немного случайности (exploration), чтобы агент не застрял в локальном максимуме.

7. Связь V и Q-функций через политику

$$V_\pi(s) = \sum_a \pi(a | s) Q_\pi(s, a)$$

То есть ценность состояния — это **средневзвешенная ценность всех действий** в нём (взвешенная по вероятностям политики).

8. Пример интуитивно

Представим среду

Агент в игре Super Mario хочет добраться до финиша и собирает монеты.

Состояние	Возможные действия	Q-ценности	Выбор
Start	Jump=0.5, Run=0.6	→ выберет Run	
Middle	Jump=0.9, Run=0.7	→ выберет Jump	
Near Finish	Jump=0.1, Run=1.0	→ выберет Run	

В итоге формируется политика:

$$\pi(s) = \arg \max_a Q(s, a)$$

9. Почему вычисление value-функции трудное

Чтобы посчитать $V(s)$ или $Q(s, a)$ напрямую, нужно суммировать все возможные будущие награды для всех траекторий.

Это **экспоненциально сложная задача**, особенно когда среда стохастическая.

Пример: если у нас 10 шагов и 4 возможных действия на каждом — всего $4^{10} = 1,048,576$ возможных путей!

Невозможно вычислить аналитически — нужна рекуррентная аппроксимация. И тут на помощь приходит **уравнение Беллмана**.

10. Подготовка к следующей теме — уравнение Беллмана

Чтобы обойти прямое суммирование наград, Беллман предложил **рекуррентное определение**:

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1})]$$

и аналогично для Q-функции:

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma Q_{\pi}(S_{t+1}, A_{t+1})]$$

То есть ценность текущего состояния равна текущей награде плюс ценности следующего состояния (с учетом дисконтирования).

Это **основа для Q-Learning, SARSA, TD(0)** и даже **DQN**.

Итоговая таблица

Понятие	Формула	Интуиция
V-функция	$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$	"насколько хорошо быть в этом состоянии"
Q-функция	$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$	"насколько хорошо сделать это действие"
Greedy Policy	$\pi(s) = \arg \max_a Q(s, a)$	выбираем действие с наибольшим Q
ϵ-Greedy Policy	добавляем случайность для исследования	компромисс исследование/использование
Беллман-идея	$V = R + \gamma V'$	рекурсивное приближение ценности

Основано на:

- Sutton & Barto, *Reinforcement Learning: An Introduction* (2020)
- Hugging Face Deep RL Course, Unit 2
- Andrea Lonza, *Алгоритмы обучения с подкреплением на Python* (2020)